Decision-aligned eviction-value prediction for learning-augmented caching

Soroush Vahidi[a]

[a] *Ying Wu College of Computing, New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

Caching with predictive information remains challenging because the value of a prediction depends not only on its accuracy, but also on how it is translated into an eviction decision. We study online paging from a candidate-level perspective in which, at each full-cache miss, the policy evaluates the items currently in cache and evicts the candidate with the smallest predicted downstream harm. To support this decision, we define a finite-horizon eviction-value target that estimates the short-term cost of evicting each candidate from the current cache state using a fixed continuation rule that yields stable and reproducible supervision. We instantiate this framework in the unweighted paging setting and also describe, but do not yet empirically validate, a lightweight guarded extension that can temporarily revert to a conservative fallback policy when recent behavior indicates locally unsafe learned decisions; we present this guard as a design extension for future empirical study rather than as a validated component of the present results. The main contribution is a decision-aligned supervision target for eviction scoring together with a candidate-scoring framework built on that target. The empirical results show that short-horizon nonlinear scorers are the strongest configurations within the evaluated offline ablation and suggest that supervision-target design is a central issue in learning-augmented caching. We additionally report an end-to-end online replay evaluation across three cache capacities (32, 64, and 128 slots) over seven trace families: in this evaluation the proposed policy does not outperform LRU, SIEVE, or FIFO-Reinsertion, and its gap relative to these baselines widens non-monotonically at the largest evaluated capacity, a pattern we analyze in detail and link to the single-step continua-

Email address: sv96@njit.edu (Soroush Vahidi)

tion rule used to construct the supervision target. Overall, the findings support finite-horizon eviction-value prediction as a well-motivated and learnable supervision-target design for learning-augmented cache replacement, while showing that the present instantiation of the policy is not yet a practically superior online replacement rule relative to strong lightweight baselines.

Keywords: learning-augmented caching, online paging, cache replacement, eviction-value prediction, caching baselines

---

1. Introduction

1.1. Problem Setting and Motivation

Caching is a fundamental systems mechanism for reducing latency, lowering backend load, and improving throughput in data-intensive services. In online caching, requests arrive sequentially, and when a miss occurs under full capacity, the policy must choose an eviction without access to future requests. Even in the classical paging setting, this local choice has important downstream consequences: a poor eviction can create avoidable future misses, whereas a better choice can preserve useful locality and reduce cumulative cost [1, 2].

Recent work on learning-augmented caching has shown that predictive information can improve online performance when it is informative and incorporated carefully into the replacement rule [3, 4, 5]. At the same time, this literature also makes clear that predictive gains are highly regime dependent. Methods that perform well under one workload, cache size, or prediction model may degrade substantially when predictions are noisy, shifted, or weakly aligned with the actual replacement objective [6, 7]. More broadly, this reflects a central theme in algorithms with predictions: predictive information is useful only when it improves the decision that the online algorithm must actually make [8].

This observation is especially important for practical cache control. At a full-cache miss, the relevant question is not merely whether a predictor should be trusted in the abstract, but which currently cached item is safest to remove. The replacement problem is therefore inherently a candidate-selection problem, and from this perspective the learning target matters. If learning is organized only around a coarse proxy, such as a trust signal or another indirect future hint, the resulting policy may remain weakly aligned with the eviction decision itself. By contrast, a candidate-level target asks

a more direct question: what downstream harm is associated with evicting each available item under the current decision context?

This paper studies that candidate-centric view, modeling each full-cache miss as a structured comparison among the current eviction candidates rather than as a single global trust decision about a predictive hint. Section 1.2 states the resulting research objective and the three aims that follow from it.

1.2. Research Objective and Scope

Building on the candidate-centric premise introduced above, the objective of this study is to investigate a learning-augmented caching method whose supervision target is aligned with the eviction decision itself: the policy estimates the short-horizon downstream cost of evicting each item currently in cache and selects the candidate with the smallest predicted cost.

Within this perspective, the paper has three aims. First, it formulates eviction as a candidate-level value-prediction problem in the unweighted paging setting. Second, it develops an empirical framework for evaluating this decision rule against classical, predictive, and robust reference policies under a common replay protocol. Third, it studies whether lightweight fallback mechanisms can provide additional protection when learned decisions appear locally unsafe. Taken together, these aims position the work as a methodological and empirical study of decision-aligned cache control.

The scope of the paper is intentionally focused. We study standard online paging with fixed cache capacity and unit miss cost because this setting isolates the eviction-choice problem cleanly and supports direct comparison with established reference policies. Broader caching models, richer cost structures, and more ambitious control variants are important directions, but they lie outside the central scope of the present manuscript.

The paper is also deliberate in what it claims. We do not present a new competitive-ratio guarantee for the learned policy, and we do not claim that the proposed target universally outperforms all alternative learning-augmented formulations. Instead, the contribution is to define a more decision-aligned supervision target, instantiate a candidate-scoring framework based on that target, and examine the empirical conditions under which this approach is useful. In this sense, the paper is best understood as an empirical and methodological contribution on how predictive context can be translated into more decision-aligned online eviction choices.

More broadly, the study treats cache replacement as an intelligent sequential decision problem in which prediction, local action scoring, and practical

robustness must work together. The resulting framework is intended as a controlled step toward more reliable learning-augmented cache management rather than as a complete resolution of the broader caching problem.

1.3. Main Contributions

The main contributions of this paper are as follows.

1. We formulate learning-augmented paging from a candidate-level decision perspective, in which each full-cache miss is treated as a structured comparison among the items currently stored in the cache. This reframes the use of predictive information around the quality of concrete eviction actions rather than around global trust in advice alone.
2. We introduce an eviction-value prediction framework for online paging that uses short-horizon downstream replay to construct a decision-aligned supervision target for each eviction candidate. The resulting target is intended to model the local downstream harm of an eviction choice more directly than advice-only, action-imitation, or indirect proxy formulations.
3. We provide a reproducible empirical framework for studying the proposed method against classical, predictive, and robust reference policies under common replay semantics, with explicit separation between offline scorer analysis and downstream online replay evaluation.
4. We contribute an empirical and methodological perspective on decision-aligned cache replacement that highlights supervision-target design as a central issue in learning-augmented caching. In particular, the paper provides evidence that candidate-level finite-horizon eviction scoring is a plausible and practically grounded direction for integrating predictive context into online eviction decisions under imperfect predictive information.

1.4. Related Work

The classical foundation of this problem lies in online paging and cache replacement, where eviction decisions must be made without access to future requests. Belady's offline rule provides the standard clairvoyant benchmark, while competitive paging established the canonical worst-case framework for reasoning about online replacement quality [1, 2]. These works capture the central difficulty that remains in modern cache control: the quality of an

eviction decision depends on future reuse, yet the decision itself must be made online.

Learning-augmented caching extends this setting by allowing the online algorithm to exploit predictive information. In the paging literature, this line has focused primarily on advice such as next-arrival predictions, predicted actions, and limited future-oriented queries, together with robustness guarantees when the advice is inaccurate [3, 9, 10, 5]. Closely related work on untrusted predictions and robust online algorithms studies how predictive decisions can be combined with safer fallback behavior or expert-style switching without sacrificing worst-case protection [4, 6, 7, 11]. Our work is aligned with this literature in its concern for robustness, but differs in emphasis: rather than centering the design on trust in a hint or on policy-level switching alone, we study a candidate-level supervision target tied directly to the eviction decision itself.

A second relevant line comes from learned cache-replacement systems that use future-derived supervision to guide eviction. PARROT learns to imitate Belady-style eviction behavior from trace-derived oracle decisions, while LRB and Raven use Belady-guided future-access prediction to drive learned replacement decisions [12, 13, 14]. Mockingjay moves beyond binary Belady-style labels by learning finer-grained reuse-time estimates for eviction ordering [15]. HALP is especially relevant because it uses candidate-level preference learning from future re-access outcomes rather than direct action imitation [16]. These papers are close to the present work in that they move eviction learning beyond purely hand-designed heuristics. However, they are still organized around oracle imitation, next-arrival or reuse-distance prediction, or pairwise future preference signals, rather than explicit supervision on finite-horizon candidate-specific downstream harm. A faithful empirical reimplementation of HALP is outside the scope of the present revision; the comparison above is therefore analytical rather than empirical, and we treat this as an explicit scope limitation rather than an omitted result.

This distinction matters because, in many prior formulations, the learned quantity remains an intermediate proxy that must still be translated into an eviction action. Such proxies include oracle action labels, reuse-distance estimates, next-arrival forecasts, and pairwise candidate orderings [12, 15, 14, 16]. By contrast, our formulation is motivated by the view that eviction is inherently a local action-selection problem, so the supervision target should approximate the downstream consequence of each available action as directly as possible. In this sense, the paper is conceptually closer to decision-focused

learning and predict-then-optimize perspectives, even though those frameworks have not been developed explicitly for cache-replacement supervision in the paging literature [17, 18].

The paper is also related to recent finite-horizon predictive approaches such as MUSTACHE, which uses multi-step-ahead page prediction to improve eviction decisions [19]. That work is particularly relevant because it introduces an explicit forecasting horizon. Our method differs in what is supervised: rather than predicting future pages or arrival times and then deriving eviction decisions from those forecasts, we train directly on a finite-horizon eviction-cost target defined at the candidate level.

Our empirical comparison also includes lightweight structural baselines from the CLOCK / Second-Chance family. SIEVE is a recent turn-key eviction algorithm that maintains a single FIFO queue together with one visited bit per resident object and an advancing hand pointer, giving retained objects a second chance without moving them, and it has been shown to match or exceed LRU's efficiency on large-scale web-cache traces [20]. FIFO-Reinsertion belongs to the same algorithmic family but instead physically reinserts a retained survivor at the head of the queue rather than advancing a hand; this contrast between hand-advancement and physical reinsertion is exactly the distinction used to position SIEVE against the broader CLOCK / Second-Chance / FIFO-Reinsertion family in [20]. We include both as non-learned, low-overhead reference points in our empirical comparison, since they represent strong and practically relevant baselines against which to assess the value of learned candidate-level scoring.

Finally, the empirical side of the study connects to prior work on realistic trace families and production-oriented cache workloads. The repository associated with this paper supports heterogeneous public and production-style traces, including BrightKite, SPEC CPU2006, Twitter cache-cluster workloads, and SHARDS-related settings [21, 22, 23, 24]. Additional sources in the broader pipeline include Citi Bike, Wikimedia pageviews, and open cache-workload resources [25, 26, 27, 28]. These are not presented as new datasets. Rather, they provide replay environments for evaluating learned eviction mechanisms across different workload regimes.

2. Main Method

2.1. System Model and Preliminaries

We study online caching in the standard unweighted paging setting. Let

$$\sigma = (r_1, r_2, \ldots, r_T) \tag{1}$$

denote the request sequence, where each request $r_t \in \mathcal{P}$ refers to an item in a finite universe $\mathcal{P}$. The cache has capacity k, and its content immediately before serving request r_t is denoted by $C_t \subseteq \mathcal{P}$ with $|C_t| \leq k$. If $r_t \in C_t$, the request is a hit; otherwise, it is a miss. When a miss occurs and the cache is full, that is, when $|C_t| = k$, the policy must evict one cached item before inserting r_t [1, 2].

We focus on the unweighted setting because it isolates the eviction-choice problem while preserving clean comparison with classical, predictive, and robust baselines. Each miss incurs unit cost, so performance is measured primarily by total misses. If $m_t \in \{0, 1\}$ denotes the miss indicator at time t, then the total cost over the sequence is

$$\mathrm{Cost}(\sigma) = \sum_{t=1}^{T} m_t. \tag{2}$$

Learning-augmented caching enriches this online setting with predictive information that may help guide replacement decisions. Prior work has considered advice in the form of predicted next-arrival times, predicted cache states, and limited access to future-oriented signals [3, 4, 5]. In this paper, such information is treated as auxiliary decision-time context rather than as an instruction that should be followed directly. The goal is therefore not only to assess whether a predictor is trustworthy, but to determine how available predictive context should influence the eviction decision at a full-cache miss.

Formally, when a miss occurs at time t and $|C_t| = k$, the eviction candidates are the items currently stored in the cache:

$$\mathcal{V}_t = C_t. \tag{3}$$

The online action is to select some $q \in \mathcal{V}_t$ for eviction. Classical policies define this action through fixed rules such as recency or marking, whereas predictive policies often derive it from a forecast or advice signal [2, 3]. Our

perspective is different: we evaluate an eviction by its downstream effect on future misses.

To make this idea operational, we define a finite-horizon candidate loss. For a candidate $q \in \mathcal{V}_t$ and horizon $H \geq 1$, let

$$L_H(q,t) = \sum_{\tau=t+1}^{t+H} m_\tau^{(q,t)} \tag{4}$$

denote the number of misses incurred over the next H requests after forcibly evicting q at time t and replaying the subsequent cache evolution under a fixed continuation rule. Here, $m_\tau^{(q,t)} \in \{0,1\}$ is the miss indicator at time τ under that counterfactual trajectory. In the implementation studied here, the continuation rule is a lightweight LRU-style replay. This yields a deterministic and computationally tractable supervision target. It should be interpreted as a local proxy for the downstream harm of an eviction choice, not as an offline-optimal objective.

This candidate-level formulation is the central shift of the method. Instead of asking only whether a predictive signal should be trusted, we ask which currently cached item appears least costly to remove under the present state and context. The remainder of the method section builds on this formulation to define the learned scoring rule and then introduces practical fallback behavior for settings in which learned decisions appear unreliable.

2.2. Eviction-Value Prediction Framework

The central idea of the proposed method is to replace coarse trust-oriented control with a direct estimate of *eviction quality* at decision time. Consider a miss at time t when the cache is full, so that the candidate set is $\mathcal{V}_t = C_t$. Instead of asking only whether a predictive signal should be followed, the method assigns each candidate $q \in \mathcal{V}_t$ a score that estimates the downstream harm of evicting that candidate under the current context. The selected victim is then the candidate with the smallest predicted loss.

Let $H \geq 1$ denote a finite replay horizon. For a candidate $q \in \mathcal{V}_t$, define its horizon-H eviction loss by

$$L_H(q,t) = \sum_{\tau=t+1}^{t+H} m_\tau^{(q,t)}, \tag{5}$$

where $m_\tau^{(q,t)} \in \{0,1\}$ is the miss indicator at future step τ under a counterfactual trajectory in which q is evicted at time t and the subsequent cache

evolution is replayed under a fixed continuation rule. In the current implementation, the default continuation rule is a lightweight LRU-style replay. This choice yields a deterministic and computationally tractable supervision signal and serves as a practical approximation rather than a claim of optimal continuation. Alternative continuation rules are treated as exploratory sensitivity directions rather than part of the main method.

The quantity in (5) is not an offline optimum. Rather, it is a local proxy for the downstream harm of an eviction choice. This is the key modeling shift of the framework. Prior learned caching methods often predict a proxy such as next arrival, reuse distance, an oracle-derived action, or a pairwise preference. Here the supervision target is tied more directly to the actual online decision: among the currently cached items, which one appears least costly to remove over a short lookahead window?

At each decision time, the framework constructs a feature vector for every candidate $q \in \mathcal{V}_t$,

$$x(q,t) \in \mathbb{R}^d, \tag{6}$$

where the features summarize the current request, the cache state, and candidate-specific predictive context. In the current repository, these features are organized into a small number of interpretable groups, including request-side features, candidate-side recency and reuse features, predictor–LRU comparison features, cache-state summary features, disagreement features, and recent-activity features. When predictive metadata are available, they are incorporated through these feature groups rather than treated as hard instructions. Table 1 summarizes the feature families used by the scorer.

The learned scorer is a function

$$\widehat{L}_H(q,t) = f_\theta(x(q,t)), \tag{7}$$

and the eviction decision becomes

$$q_t^* = \arg\min_{q \in \mathcal{V}_t} \widehat{L}_H(q,t). \tag{8}$$

This candidate-centric formulation has two practical advantages. First, it is directly aligned with the actual online choice made at a full-cache miss: the policy compares the available candidates and selects one victim. Second, it supports robust extensions without changing the basic decision object. If the learned scores become unreliable in a particular regime, a fallback mechanism can still intervene at the same decision point by overriding or temporarily replacing the learned victim choice.

The framework should therefore be understood as a decision-focused layer over online caching rather than as a claim of offline optimality. The replay target in (5) provides a stable and operational route for integrating predictive information into cache replacement, and it forms the basis for the guarded fallback mechanism described next.

Table 1: Decision-time feature groups used by the eviction-value scorer. The table summarizes the main feature families implemented in the current artifact pipeline; the full feature specification is repository-based and reproducibility-oriented.

Feature group	Role in decision-time scoring
Request-side features	Encode properties of the current request and its immediate decision context.
Candidate-side reuse and recency features	Summarize candidate age, recent reuse behavior, and local history relevant to eviction risk.
Predictor–LRU comparison features	Capture how predictive context differs from recency-based signals for the same candidate.
Cache-state summary features	Describe the current cache composition and local competition among candidates.
Disagreement features	Indicate whether predictor-oriented and recency-oriented views point to different eviction choices.
Recent-activity features	Summarize short-window request activity and local workload dynamics near the decision point.

2.3. Guarded Fallback Mechanism

A learned eviction scorer can improve decision quality when its candidate rankings are well aligned with future reuse, but its behavior may deteriorate when the workload changes or when local predictive context becomes unreliable. We therefore describe a lightweight optional guard that can temporarily delegate to a conservative fallback when recent online behavior suggests locally unsafe learned decisions; we treat this guard as an implementation safeguard for future empirical study rather than as a demonstrated component of the present results (see Limitations).

The guard uses repeated *very-early-return* evictions as its signal. Let q_t^* denote the victim proposed by the learned scorer at time t. If the evicted item is requested again within a short window, the decision is treated as

suspicious because it indicates that an item with immediate reuse value may have been removed.

Formally, let

$$E_t = \begin{cases} 1, & \text{if the item evicted at time } t \text{ returns within } W \text{ subsequent requests,} \\ 0, & \text{otherwise,} \end{cases} \tag{9}$$

where W is a small early-return window. The mechanism aggregates these suspicious events over a sliding interval of length T. Writing the recent suspicious count as

$$S_t = \sum_{\tau=\max(1,t-T+1)}^{t} E_\tau, \tag{10}$$

the trigger condition is

$$S_t \geq M, \tag{11}$$

where M is the threshold required to enter fallback mode.

Once triggered, the controller temporarily delegates the eviction choice to a conservative fallback policy for a bounded duration D. In the current implementation, the fallback is configurable and may be instantiated by a simple baseline such as least-recently used eviction or a marker-style policy. Let $\mathcal{M}_t \in \{\textsc{base}, \textsc{fallback}\}$ denote the operating mode at time t. The final victim is then

$$\widetilde{q}_t = \begin{cases} q_t^*, & \text{if } \mathcal{M}_t = \textsc{base}, \\ q_t^{\mathrm{fb}}, & \text{if } \mathcal{M}_t = \textsc{fallback}, \end{cases} \tag{12}$$

where q_t^{fb} is the victim selected by the fallback controller. After D requests in fallback mode, the system returns to base mode unless the trigger condition fires again.

This mechanism is intentionally lightweight and heuristic: trigger signal, thresholds, duration, and fallback choice are implementation decisions for controlled study, not a theorem-backed robustness guarantee [6, 7]. The learned scorer proposes a victim; the optional guard may temporarily hand control to a conservative fallback when local evidence indicates repeated unsafe decisions. The guard's end-to-end effect has not been measured in the present study (see Limitations).

2.4. Algorithmic Workflow and Implementation Details

This subsection summarizes how the proposed method is instantiated in the experimental framework. The workflow is modular: trace preparation, candidate-level dataset construction, model fitting, online replay, and optional fallback control are implemented as separate components within a common replay environment. This separation keeps the learned policy, the reference baselines, and the evaluation pipeline aligned under the same execution semantics. Figure 1 provides a high-level overview of this workflow, and the five stages below summarize the guarded online decision procedure.

At a high level, the method proceeds in five stages.

1. **Trace preparation.** Public and manifest-based traces are converted into a common internal format. These processed request streams are then replayed under fixed cache capacities to construct decision points and support downstream evaluation.
2. **Candidate-level example construction.** At each full-cache miss, the current eviction candidates are enumerated and one supervised example is created per candidate. Each example contains decision-time features together with a finite-horizon replay label derived from the downstream miss count after forcibly evicting that candidate.
3. **Model fitting.** A supervised model is trained to predict the candidate loss target. In the present study, model selection is performed across replay horizons and model families, with nonlinear scorers emerging as the strongest configuration in the offline ablation.
4. **Online eviction replay.** During execution, the scorer evaluates all current candidates at each full-cache miss, predicts their losses, and evicts the candidate with minimum predicted loss.
5. **Optional fallback control.** When the guarded variant is enabled, the controller monitors suspicious early-return events and temporarily switches to a conservative fallback policy when recent online behavior suggests that the learned scorer is locally unsafe.

In the implementation studied here, candidate-level supervision is generated for the main eviction-value line, a scorer is trained for a chosen replay horizon, and the resulting policy is evaluated in the same simulator used for the reference baselines. The framework records aggregate metrics together with step-level logs so that downstream behavior can be analyzed jointly with fallback activity.

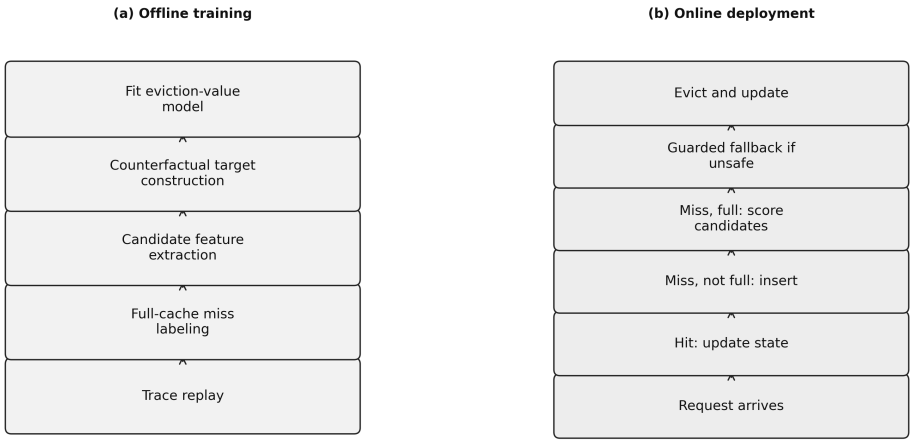

Figure 1: Conceptual overview of the eviction-value workflow. Offline replay is used to construct candidate-level training examples and finite-horizon loss targets, a supervised scorer is fitted to estimate decision-time eviction loss, and the online policy evicts the candidate with minimum predicted loss. An optional lightweight fallback layer can temporarily delegate control to a conservative baseline when repeated local signals indicate unsafe learned decisions.

At decision time t, the method first builds the candidate feature matrix

$$X_t = \begin{bmatrix} x(q_1, t)^\top \\ x(q_2, t)^\top \\ \vdots \\ x(q_k, t)^\top \end{bmatrix}, \tag{13}$$

where $\{q_1, \ldots, q_k\} = C_t$ is the current candidate set. The learned scorer then returns the vector of predicted losses

$$\widehat{\ell}_t = \begin{bmatrix} \widehat{L}_H(q_1, t) \\ \widehat{L}_H(q_2, t) \\ \vdots \\ \widehat{L}_H(q_k, t) \end{bmatrix}, \tag{14}$$

and the learned victim is selected by

$$q_t^* = \arg\min_{q_i \in C_t} \widehat{L}_H(q_i, t). \tag{15}$$

If the optional fallback controller is active, this proposal is passed through the mechanism described in the previous subsection before the final eviction is applied.

Several implementation choices matter for interpreting the results. First, the replay label is intentionally local and finite-horizon rather than globally optimal, because the goal is to obtain a stable and reproducible decision proxy from observable cache states. Second, learned policies and baselines are executed in the same simulator with the same trace-loading interface, which keeps replay semantics consistent across methods. Third, the framework records diagnostics for both unguarded and guarded variants, including mode switches, trigger counts, time spent in fallback mode, and step-level decision logs. These diagnostics help separate the behavior of the scorer itself from the behavior induced by the fallback layer.

Finally, the implementation is designed to support conservative empirical claims. The main methodological contribution of the present manuscript is the candidate-level supervision and scoring framework. Other policy families and exploratory variants in the broader codebase are used primarily for contextual comparison and sensitivity analysis rather than as the center of the paper.

3. Experiments and Discussion

3.1. Experimental Setup

The experiments are designed to evaluate the proposed eviction-value framework under a controlled replay protocol. All policies are executed within a common simulator using the same processed traces, cache capacities, request ordering, and trace-loading interface. This unified execution setting is important because it reduces confounding from preprocessing differences, trace-format inconsistencies, and policy-specific replay semantics. Consequently, observed differences in performance can be attributed more directly to differences in eviction behavior.

The empirical focus is the main candidate-scoring policy, `evict_value_v1`, together with a selected set of classical, predictive, and robust reference policies from the learning-augmented caching literature [3, 4, 6, 7]. These baselines are included because they represent the most relevant practical comparison points for online paging under imperfect predictive information. Other policy families implemented in the broader codebase are treated as ex-

ploratory or contextual comparisons and are not part of the main empirical claim unless their results are explicitly reported.

The evaluation has two complementary components. The first is *offline scorer construction and selection*. In this component, candidate-level examples are generated from replayed eviction decisions, finite-horizon loss labels are constructed, and model families are compared using target-quality metrics that assess how well the learned scorer approximates the proposed supervision objective. The second is *online replay evaluation*, in which the induced policy is executed in the same simulator as the reference baselines and compared through downstream replay performance. This separation is methodologically important because strong performance on an offline proxy does not necessarily translate into the best online replacement decisions once the scorer is embedded in the full decision loop.

To isolate the eviction-choice problem, the study is conducted in the standard unweighted paging setting with fixed cache capacity and unit miss cost. The principal downstream metric is therefore the total number of cache misses over the replayed request sequence. This is the most direct operational metric in the present setting because each miss contributes equally to the objective. In addition to aggregate summaries, the framework records step-level outputs and policy-specific diagnostics so that one can examine not only final performance but also intermediate behavior such as candidate disagreements, early-return events, and, when applicable, fallback activity in guarded variants.

The implementation also distinguishes between full trained-model evaluation and lightweight surrogate execution paths. This distinction is useful because it separates the behavior of the policy family from the availability of a specific stored model artifact and allows controlled sanity checks when full trained artifacts are not the object of analysis. In the present manuscript, the strongest displayed evidence concerns scorer selection, including replay-horizon and model-family comparisons. Broader end-to-end replay comparisons are interpreted only when the corresponding evaluation outputs are explicitly shown.

Overall, the experimental setup is intended to be reproducible at the level of scripts, replay configuration, and generated outputs. For this reason, the empirical section is organized around systematically generated tables, figures, and machine-readable artifacts rather than one-off manual summaries. At the same time, the presentation remains deliberately conservative: claims are restricted to what is directly supported by the reported evaluation.

3.2. Datasets, Baselines, and Evaluation Metrics

The empirical study is organized around a common replay interface applied to heterogeneous request traces and a baseline pool chosen to reflect the most relevant practical comparison points for learning-augmented paging. For manuscript purposes, it is important to distinguish clearly between *repository-supported* workload families and the *artifact-backed subset* used in the reported evaluation. The repository contains broader data-preparation and experiment infrastructure than any single manuscript table should claim. Accordingly, the quantitative results discussed in this paper are restricted to the traces, capacities, and policy comparisons that are directly supported by the canonical manuscript artifacts generated from the designated evaluation pipeline.

The broader repository supports public and manifest-based trace families including BrightKite, Citi Bike, SPEC CPU2006, Wikimedia pageviews under the `wiki2018` schema, and production-style cache traces such as Twemcache, MetaKV, MetaCDN, and CloudPhysics-related workloads [21, 25, 22, 26, 23, 24, 27, 28]. These sources are not presented as new datasets. Rather, they provide replay environments for studying learned eviction mechanisms under varied locality structures and workload regimes. In the manuscript, however, we report only the subset that is directly instantiated by the canonical evaluation artifacts, and we avoid treating repository support for additional traces as evidence unless the corresponding results are explicitly shown.

The baseline pool is selected to cover four comparison roles: classical paging references, CLOCK-family structural references, predictive paging references, and robust learning-augmented controllers. Classical and CLOCK-family references include least-recently used eviction, SIEVE, and FIFO-Reinsertion, which remain essential lightweight anchors for unweighted online caching [1, 2, 20]. Predictive and robust references include predictive marker, trust-and-doubt-style control, a regret-driven selective-trust gating controller (REST), and the blind-oracle/LRU combiner [3, 4, 6, 7, 5, 9]. These methods are used because they represent strong and practically relevant reference points when predictions are informative but imperfect.

Within the learned-policy family, the main method studied in this paper is `evict_value_v1`, which predicts a candidate-specific finite-horizon downstream eviction loss and evicts the candidate with minimum predicted loss. The framework also implements an optional guard that can temporarily defer to a conservative fallback policy when recent online behavior indicates locally unsafe learned decisions; this guard is an implementation safeguard

evaluated only as an ablation in the present study, and we do not claim that it improves end-to-end miss ratio unless future experiments support it. Other policy families implemented in the repository, including a standalone marker-style policy, a robust follow-the-prediction combiner with marker fallback, pairwise-ranking variants, sentinel-style guards, and additional exploratory controllers, are retained for contextual analysis and ongoing research development, but they are not treated as central baselines for the present manuscript unless their results are explicitly reported in the canonical artifact bundle.

Table 2 summarizes the workload families relevant to the present study from a manuscript perspective. The table is intentionally phrased at the family level to separate source provenance from the narrower question of which canonical artifacts are currently reported. Table 3 summarizes the main policy families used in the empirical comparison.

Table 2: Trace families in the artifact-backed evaluation. The repository supports additional workloads; reported results are restricted to the canonical artifact subset.

Trace family	**Representative source**	**Role in this work**
BrightKite	BrightKite mobility trace [21]	Public request stream with heterogeneous locality structure for controlled replay evaluation.
Citi Bike	Citi Bike system data [25]	Public event stream used to derive replayable request sequences under a common preprocessing interface.
SPEC CPU2006	SPEC CPU2006 benchmark descriptions [22]	Classical systems-style trace family providing a contrasting locality regime.
Wikimedia pageviews	Wikimedia pageview dumps [26]	Large public page-access source for replay-based cache evaluation.
Production-style cache traces	Twitter cache clusters, SHARDS-related resources, and open cache-workload sources [23, 24, 27, 28]	Manifest-based or documented production-style workloads used to broaden workload diversity in the repository and, when artifact-backed, in the manuscript evaluation.

The primary evaluation metric is the total number of cache misses in-

Table 3: Main policy families in the empirical comparison. A diagnostic-only blind-oracle variant used for internal sanity checks is excluded from the reported comparisons.

Label	**Category**	**Role in evaluation**
LRU	Classical baseline	Standard recency-based eviction reference [1].
SIEVE	Classical baseline	CLOCK-family turn-key eviction algorithm using a single FIFO queue and an advancing hand pointer over retained objects [20].
FIFO-Reinsertion	Classical baseline	CLOCK / Second-Chance family baseline that reinserts a retained survivor at the head of the queue rather than advancing a hand [20].
PredMk	Predictive baseline	Prediction-aware marker-style paging baseline [3].
BO/LRU	Combiner baseline	Blind-oracle/LRU practical combiner baseline [6].
T&D	Robust baseline	Trust-and-doubt-style controller for untrusted predictions [4].
REST	Heuristic baseline	Regret-driven selective-trust gating controller that switches between a predictor-driven eviction rule and an LRU fallback based on delayed online feedback.
EV	Proposed method	Candidate-level finite-horizon eviction-value predictor.

curred during online replay. This is the natural objective in the unweighted paging setting because each miss carries unit cost, so replay misses directly measure downstream replacement quality. We also report hit rate and, where useful for interpretation, miss deltas relative to the LRU baseline. For offline scorer analysis, we additionally use target-quality diagnostics such as regret-style error summaries, Top-1 agreement, and disagreement-oriented statistics. These auxiliary quantities are useful for understanding the learned scorer, but they remain secondary to downstream replay performance.

This evaluation design is intended to answer three questions. First, can candidate-level eviction-value prediction improve upon strong classical and predictive reference policies under a common replay protocol? Second, how does the learned policy compare with robust combiner-style baselines when predictive information is imperfect? Third, how do offline target-quality trends relate to downstream online replay behavior? In the present manuscript, the strongest displayed evidence concerns the scorer-selection and horizon-analysis path, while broader multi-trace replay claims are restricted to the canonical artifact-backed evaluation outputs. This distinction is important for maintaining consistency between the manuscript narrative and the repository evidence.

3.3. Offline Ablation and Model Selection

We begin with the offline ablation used to select the scorer configuration for `evict_value_v1`. Table 4 and Fig. 2 summarize validation and test regret across replay horizons and model families. These results are used to characterize the learnability of the candidate-level target and to motivate the scorer configuration carried into the main policy line.

Two patterns are clear. First, the nonlinear tree-based models are consistently stronger than the linear ridge baseline. Across all three replay horizons, ridge incurs substantially larger regret than either `hist_gb` or `random_forest`. This indicates that the eviction-value target is not well captured by a simple linear approximation and benefits from a more flexible scorer.

Second, shorter replay horizons are stronger in the current artifact set. Horizon $H = 4$ gives the best validation regret for `hist_gb` and the best test regret for `random_forest`, while performance deteriorates as the horizon increases to 8 and 16. This trend is consistent with the intended role of the target: a shorter horizon appears to provide a cleaner and more learnable

proxy for local eviction harm, whereas longer horizons introduce additional downstream interactions that are harder to model accurately.

The Top-1 results support the same interpretation. On both validation and test splits, the tree-based models achieve lower Top-1 error than ridge across the evaluated horizons, suggesting that nonlinear scorers preserve the candidate ordering needed for eviction decisions more effectively than a linear baseline. At the same time, the gap between `hist_gb` and `random_forest` is modest relative to their shared advantage over ridge. The main methodological conclusion is therefore about the value of nonlinear scoring and short-horizon supervision, not about one tree-based model being universally dominant.

Taken together, these ablation results support a conservative model-selection conclusion: within the current artifact-backed evidence, the candidate-level eviction-value framework is strongest with a short replay horizon and a nonlinear scorer. This evidence does not by itself establish superiority in end-to-end online replay, but it does show that the proposed target yields a meaningful and learnable signal for downstream eviction scoring.

Table 4: Offline ablation for `evict_value_v1` across replay horizons and model families. Lower regret is better, and lower Top-1 error is better.

Horizon	Model	Val. regret	Test regret	Val. Top-1	Test Top-1
4	hist_gb	0.0091	0.0055	0.0174	0.0316
4	random_forest	0.0134	0.0046	0.0154	0.0264
4	ridge	0.0649	0.0150	0.0503	0.0832
8	hist_gb	0.0203	0.0104	0.0157	0.0362
8	random_forest	0.0240	0.0120	0.0177	0.0267
8	ridge	0.0880	0.0384	0.0377	0.0795
16	hist_gb	0.0394	0.0196	0.0203	0.0365
16	random_forest	0.0431	0.0178	0.0157	0.0249
16	ridge	0.1166	0.0850	0.0277	0.0826

3.4. End-to-End Online Replay Evaluation Across Available Capacities

In addition to the offline ablation above, we report end-to-end online replay results for all eight policies in Table 3, evaluated under the common simulator described in Section 3.1 across the seven trace families in Table 2, at 50,000 requests per trace. At the time of writing, this evaluation has been completed at cache capacities 32, 64, and 128 slots; capacity 256 has not

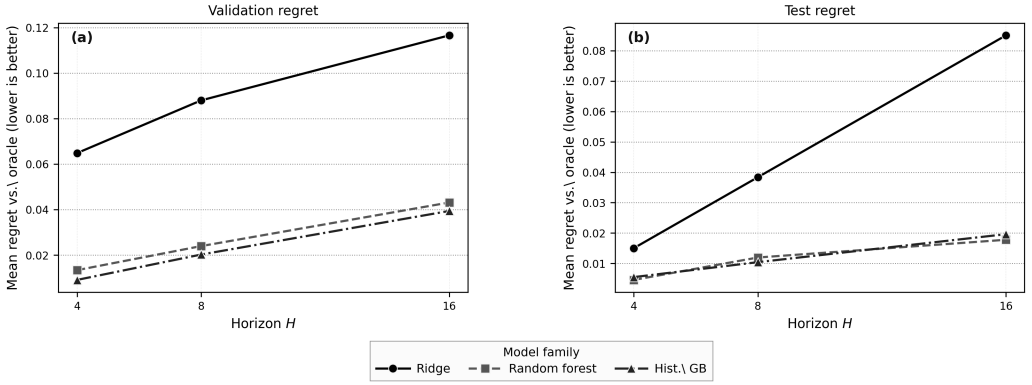

Figure 2: Offline ablation for `evict_value_v1` across replay horizons and model families. The figure summarizes validation and test regret for the configurations shown in Table 4. Lower is better.

yet been evaluated end-to-end and is not reported in this paper. We are explicit about this boundary because the capacity-128 result discussed below is itself non-monotonic relative to capacities 32 and 64, and we judged it more important to report and analyze the existing three-point trend honestly than to extend further computation into a larger capacity sweep before doing so.

Table 5 reports mean replay misses, averaged across the seven trace families, for each policy at each evaluated capacity, and Fig. 3 visualizes the same data together with `evict_value_v1`'s relative gap against the three strongest lightweight baselines (LRU, SIEVE, and FIFO-Reinsertion).

Table 5: End-to-end online replay evaluation: mean misses across the seven trace families in Table 2 (50,000 requests/trace), at each evaluated cache capacity. Capacity 256 has not yet been evaluated end-to-end.

Policy	Cap. 32	Cap. 64	Cap. 128
LRU	34667.1	33650.1	32495.6
SIEVE	35313.6	34362.7	33279.3
FIFO-Reinsertion	34688.0	33672.1	32530.6
PredMk	34843.9	33773.1	32664.6
T&D	35087.0	33998.7	32928.9
BO/LRU	34668.6	33651.3	32496.9
REST	34667.1	33650.1	32495.6
EV (proposed)	36344.1	34409.0	36316.6

Three observations follow directly from this available-capacity replay.

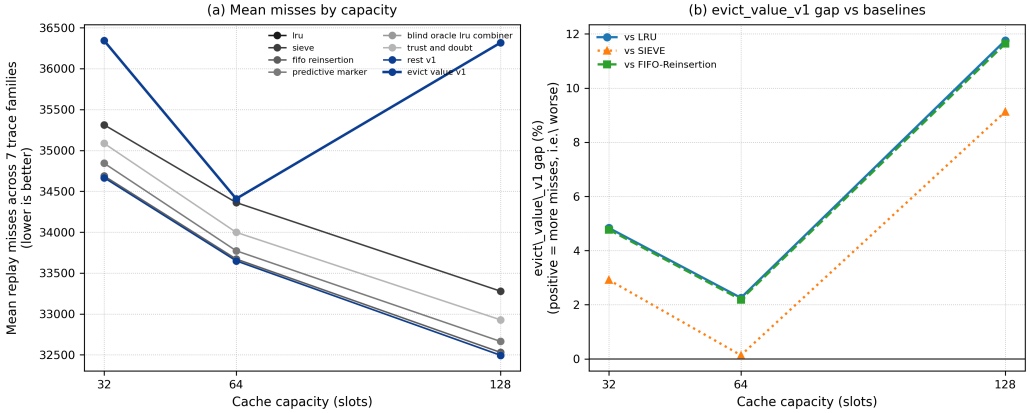

Figure 3: Capacity-trend comparison across the available evaluated capacities (32, 64, 128; capacity 256 not evaluated in this revision). (a) Mean replay misses per policy at each capacity, averaged across the seven trace families. (b) `evict_value_v1`'s relative miss gap against LRU, SIEVE, and FIFO-Reinsertion at each capacity; positive values indicate more misses than the baseline. The LRU and FIFO-Reinsertion gap curves nearly coincide because the two baselines have nearly identical miss counts in the averaged replay; distinct line styles and markers are used to keep them visually separable. The non-monotonic widening at capacity 128 is discussed in Section 3.6 and Section 4.3.

First, `evict_value_v1` does not improve on LRU, SIEVE, FIFO-Reinsertion, or REST on average at any of the three evaluated capacities: its mean-miss gap relative to LRU is +4.84% at capacity 32, +2.26% at capacity 64, and +11.76% at capacity 128 (equivalently +2.92%, +0.13%, and +9.13% against SIEVE, and +4.77%, +2.19%, and +11.64% against FIFO-Reinsertion). REST is numerically tied with LRU at all three evaluated capacities, so the same average gap applies there as well. This is a negative end-to-end result for the proposed policy relative to the strongest baselines in our pool, and we report it directly rather than restricting the paper's empirical claims to the offline ablation evidence above.

Second, the other reference policies cluster tightly around LRU at every evaluated capacity. `blind_oracle_lru_combiner` (BO/LRU) and `rest_v1` (REST) are numerically indistinguishable from LRU (0.00% gap) at all three capacities, FIFO-Reinsertion stays within 0.11% of LRU throughout, and SIEVE, T&D, and PredMk remain within roughly 0.4–2.4% of LRU at every capacity. `evict_value_v1` is the only policy in the comparison whose gap exceeds 5% at any evaluated capacity, and the only one whose relative position worsens between capacity 64 and capacity 128 rather than tracking the

otherwise stable ordering of the other seven policies.

Third, the capacity trend for `evict_value_v1` is non-monotonic: the gap narrows from capacity 32 to capacity 64 and then widens sharply at capacity 128, rather than improving or degrading smoothly with capacity. We examine this pattern at the workload level next, and return to its likely cause in the Discussion and Limitations.

3.5. Workload-Specific Breakdown

Table 6 reports `evict_value_v1`'s relative miss gap against LRU separately for each of the seven trace families at each evaluated capacity, rather than only as the family-averaged numbers in Table 5.

Table 6: `evict_value_v1` relative miss gap against LRU, by trace family and evaluated capacity. Positive values indicate more misses than LRU (worse).

Trace family	Cap. 32	Cap. 64	Cap. 128
BrightKite	+10.47%	+10.72%	+50.91%
Citi Bike	+7.90%	+4.70%	+45.49%
CloudPhysics	+0.72%	+1.57%	+3.10%
MetaCDN	+22.37%	+2.73%	+17.62%
MetaKV	+1.70%	+0.01%	+0.76%
Twemcache	+1.84%	+3.16%	+14.06%
Wikimedia[†]	+0.00%	+0.00%	+0.00%

[†]Degenerate at every evaluated capacity: all eight policies reach 50,000/50,000 misses (0% hit rate), because the working set exceeds even the largest evaluated capacity. Not informative for cross-policy comparison.

Two patterns stand out. First, Wikimedia pageviews is degenerate at every evaluated capacity, as noted in the table, and we exclude it from substantive interpretation. CloudPhysics and MetaKV remain close to LRU at every capacity (at most +3.10% and +1.70%, respectively), consistent with the family-averaged picture in Table 5.

Second, the capacity-128 widening is concentrated in, but not limited to, two families. BrightKite and Citi Bike show the largest gaps at capacity 128 (+50.91% and +45.49%, respectively), up sharply from roughly +10% or less at capacities 32 and 64. MetaCDN and Twemcache also show double-digit gaps at capacity 128 (+17.62% and +14.06%), though their capacity-32/64 trajectories differ from BrightKite's and Citi Bike's: MetaCDN's gap is actually largest at capacity 32 (+22.37%), drops to +2.73% at capacity 64,

and rises again at capacity 128, a U-shaped pattern rather than a monotonic one, while Twemcache's gap increases steadily across all three capacities (+1.84%, +3.16%, +14.06%). In total, four of the six non-degenerate families show a capacity-128 gap above 10%, while the remaining two (CloudPhysics, MetaKV) stay below 3.1% throughout. We therefore describe the capacity-128 degradation as broad-based rather than as an artifact specific to two outlier traces, while still noting that BrightKite and Citi Bike are the most extreme cases.

Taken together with the offline ablation in Section 3.3, the capacity-128 degradation documented above is broad-based across workload families rather than an artifact of a single outlier trace, reinforcing rather than qualifying away the negative end-to-end result already reported in Section 3.4. We discuss its likely cause in Section 3.6 and revisit it in Section 4.3.

3.6. Discussion and Analysis

The present results are most informative when interpreted through the central modeling choice of the paper: the proposed method is organized around a decision-aligned supervision target for candidate-level eviction scoring rather than around a global trust-calibration rule. This distinction is important because cache replacement is inherently a local comparative decision. At a full-cache miss, the policy must decide which currently cached item is least harmful to remove, not merely whether predictive information should be trusted in the abstract. From this perspective, the most direct question is not whether the advice appears accurate in isolation, but whether the learned target helps rank the available eviction candidates in a way that improves the actual online choice.

Viewed in this way, the main contribution of the current results is methodological. The offline ablation indicates that replay-based candidate supervision provides a meaningful route for translating predictive context into eviction decisions. In particular, the tree-based scorers consistently outperform the linear ridge baseline across the evaluated replay horizons, suggesting that the proposed target is learnable but not well approximated by a simple linear model. This supports the broader claim that candidate-level downstream supervision is a viable target for learning-augmented cache replacement and that the induced decision surface is sufficiently structured to benefit from nonlinear modeling.

The replay-horizon analysis provides a second useful insight. Within the current artifact set, shorter replay horizons yield the strongest regret values,

with $H = 4$ performing best among the evaluated settings. This pattern is consistent with the intended interpretation of the target. A short replay horizon appears to provide a cleaner and more stable approximation to local eviction harm, whereas longer horizons incorporate additional downstream interactions that are harder for the scorer to model reliably from decision-time features alone. In other words, the target becomes less local and therefore less learnable as the counterfactual window is extended.

More broadly, these findings suggest that supervision-target design deserves attention comparable to that given to the predictive signal itself. In the learning-augmented caching literature, discussion often centers on the accuracy of the advice or on the mechanism used to determine when that advice should be trusted. The present results point to a complementary lesson: even informative predictive context may be of limited value if the learning objective is weakly aligned with the actual eviction decision. A candidate-level downstream-loss target remains a proxy, but it is a proxy defined directly at the decision point, which gives it a clearer operational interpretation than a trust score, a generic future-prediction objective, or an indirect action surrogate.

At the same time, the interpretation of these results should remain cautious, and the end-to-end evaluation in Section 3.4 sharpens rather than removes that caution. The replay-based loss studied here is finite-horizon and local rather than globally optimal, and the offline scorer-selection ablation does not by itself establish superiority in full end-to-end online replay. The explicit end-to-end evidence we now report directly answers that question, and the answer is negative: across all three evaluated capacities, `evict_value_v1` incurs more replay misses than LRU, SIEVE, and FIFO-Reinsertion, with the gap widening sharply and non-monotonically at capacity 128. Likewise, the broader framework includes guarded variants whose end-to-end effect remains unmeasured in the present study (see Limitations). The current results therefore support the proposed supervision target and scoring formulation as a learnable and well-motivated way to pose the eviction-decision problem, but they do not support a claim of downstream competitiveness, robustness across workload regimes, or effective fallback control in deployment; on the contrary, the end-to-end evidence weighs against competitiveness at every capacity evaluated so far.

A natural question is why a learnable, decision-aligned offline target would translate into a policy that underperforms LRU, SIEVE, and FIFO-Reinsertion online, and why the gap would widen specifically at capacity

128 rather than growing smoothly with capacity. We do not have a fully confirmed causal account, but the evidence gathered to date points to a structurally plausible mechanism. The supervision target in Eq. (5) is constructed under a fixed LRU-style continuation rule: each candidate's training label assumes that, after the scored candidate is evicted, the cache continues to evolve under LRU. The deployed policy, however, recursively makes its own, generally non-LRU eviction choices at every subsequent decision. As a replay episode progresses, the policy's realized trajectory therefore increasingly diverges from the LRU-continuation assumption baked into every training label, a form of compounding distribution shift between the labeling-time counterfactual and the deployment-time trajectory. Because each eviction decision scores every one of the k resident candidates, a larger capacity gives the policy more candidates to rank per decision over the same request budget, which is consistent with, though does not by itself prove, a larger capacity amplifying this compounding effect. We discuss the supporting and limiting evidence for this account, and what would be needed to confirm it, in Section 4.3.

Taken together, the artifact-backed evidence supports a focused but mixed conclusion. Finite-horizon candidate-level eviction-loss prediction is a plausible and methodologically well-motivated direction for learning-augmented caching, and short-horizon nonlinear scorers are the strongest configuration within the offline ablation. At the current stage, this is best interpreted as evidence in favor of the target design and scoring formulation as an object of study, not as a demonstration of practical superiority over strong lightweight baselines: the end-to-end evaluation directly contradicts a superiority claim at every capacity evaluated so far. We therefore characterize `evict_value_v1`, in its present form, as a decision-aligned supervision study rather than as a practically superior online policy, while SIEVE and FIFO-Reinsertion remain strong, low-overhead baselines that the proposed method does not currently outperform. Stronger end-to-end claims and fallback-reliability claims should be reserved for future work that addresses the capacity-128 finding directly.

3.7. Overhead and Scalability

The proposed method has two distinct computational costs: an offline label-construction cost incurred once per dataset, and an online per-decision cost incurred on every full-cache miss during replay or deployment. We

report both below, separating measured wall-clock evidence from complexity analysis.

Constructing the finite-horizon eviction-value dataset used in this paper required simulating a fixed continuation policy forward from every eviction decision considered, which is inherently more expensive than constructing a supervision target that can be read directly from the trace (e.g., next-arrival time); this offline construction step took approximately 10.3 hours of wall-clock time and produced 96 GB across 662 shard files. Model training itself is comparatively inexpensive, since it operates on a bounded sample of the constructed dataset rather than on the full label set directly; fitting all evaluated horizon/model-class configurations together took approximately 7–8 minutes.

At each full-cache miss, `evict_value_v1` constructs a feature vector for every one of the k candidates currently resident in the cache and scores each with the trained model, giving $O(k)$ model-inference calls per miss, where k is the cache capacity. This contrasts with the $O(1)$-per-miss classical baselines in our comparison: LRU evicts via a single recency-ordered removal, and the CLOCK-family baselines in our pool, SIEVE and FIFO-Reinsertion, evict via an amortized $O(1)$ hand-scan or reinsertion-scan respectively, with no per-candidate model inference. This is the expected and inherent cost of candidate-level learned scoring relative to structural eviction rules, and it grows with cache capacity rather than remaining constant.

This complexity comparison is verified directly against our own implementations' source code rather than taken solely from the respective baselines' publications. We have since produced a controlled, capacity-isolated wall-clock benchmark: on a 5,000-request prefix of the BrightKite trace, measured on the author's local development machine under tmux rather than on the Wulver HPC cluster or under Slurm, the mean wall-clock cost of an `evict_value_v1` eviction decision was 75.0 ms, 152.1 ms, and 316.0 ms at capacities 32, 64, and 128, respectively – scaling nearly linearly with capacity, consistent with the $O(k)$ candidate-scoring analysis above. Over the same capacities, LRU and FIFO-Reinsertion averaged approximately 0.001 ms per timed decision, SIEVE ranged from 0.002 ms to 0.005 ms, and REST ranged from 0.04 ms to 0.18 ms, four to five orders of magnitude faster than `evict_value_v1` throughout. We report this single-trace, single-run measurement as a controlled overhead probe rather than as a claim of representativeness across all seven trace families.

3.8. Replay Horizon Selection

The choice of replay horizon H controls a tradeoff in label quality. A short horizon ties the eviction-value label closely to the immediate consequence of the scored decision, while a long horizon allows the label to absorb increasing variance from the continuation policy's own subsequent choices, effects that are downstream consequences of later decisions rather than of the eviction being scored. We observe this tradeoff empirically: validation MAE, RMSE, top-1 eviction match, and mean regret-vs-oracle all degrade monotonically as H increases from 4 to 8 to 16, for every model class evaluated. This pattern holds not only in aggregate but within every individual trace family represented in our validation sample (BrightKite, CloudPhysics, MetaCDN, Twemcache, and Wikimedia pageviews), suggesting that the advantage of shorter horizons in this framework is not specific to any one workload among those evaluated. We have not yet evaluated whether this pattern extends to the remaining trace families in our full dataset or whether it interacts with cache capacity; we flag both as open questions for future work rather than claim to have resolved them.

4. Conclusions and Future Research

4.1. Summary of Findings

This paper studied learning-augmented caching through a candidate-level view of the eviction decision. Rather than treating prediction primarily as advice to be trusted or ignored at a global level, the paper formulated each full-cache miss as a structured comparison among the items currently in cache and developed a method that predicts the finite-horizon downstream loss associated with evicting each candidate. This yields a supervision target that is more directly aligned with the online eviction decision than trust-only or indirect proxy formulations.

The current empirical evidence supports this perspective most clearly at the level of scorer construction and target selection. In particular, the offline ablation shows that the proposed eviction-value target is learnable, that nonlinear scorers are substantially stronger than a linear ridge baseline, and that shorter replay horizons provide the strongest results within the present artifact set. Taken together, these findings indicate that candidate-level downstream supervision is not merely a reformulation of the problem, but a productive object of study for learned cache replacement. We report

this distinctly from end-to-end online competitiveness, which the offline ablation alone does not establish and which we evaluate directly next.

The overall framework also includes a lightweight, optional fallback layer that can defer to a conservative baseline when recent local behavior appears unsafe; Section 4.2 and Section 4.3 discuss why this remains a design extension rather than a validated empirical contribution in the present study.

More broadly, the study suggests that supervision-target design is a central issue in learning-augmented caching. The value of predictive information depends not only on the quality of the signal, but also on whether that signal is translated into a learning objective that matches the actual online action. In this sense, the main contribution of the paper is both methodological and empirical: it introduces a candidate-level finite-horizon supervision target for eviction scoring and provides evidence that this target yields a tractable and informative learning signal for cache replacement under imperfect predictive information.

Overall, the results support a focused but mixed conclusion, discussed more fully in Section 3.6. The offline ablation indicates that finite-horizon candidate-level eviction-value prediction is a credible, well-motivated, and learnable supervision target, with short-horizon nonlinear scoring the strongest configuration. The end-to-end online replay evaluation across capacities 32, 64, and 128 is more cautionary: `evict_value_v1` does not outperform LRU, SIEVE, or FIFO-Reinsertion at any evaluated capacity, with a non-monotonic gap that widens substantially at capacity 128 (Sections 3.4 and 4.3). Stronger end-to-end claims, capacity-256 evaluation, and guarded-deployment evidence are left to future work.

4.2. Implications of the Proposed Approach

The proposed approach has several implications for learning-augmented cache design. The first is methodological: in cache replacement, the supervision target can be as important as the predictive signal itself. Much of the learning-augmented literature is organized around the quality of a hint and the question of when that hint should be trusted. The present study suggests a complementary perspective for eviction: because the online action is a structured comparison among currently cached candidates, the learning objective should be defined as close as possible to that decision.

A second implication is that robust learning-augmented caching need not be expressed only through global trust calibration or expert switching. Those ideas remain important, and robust baselines remain essential

reference points. In the present implementation, predictive information is incorporated through candidate-level scoring, and the codebase also includes an optional guard-style extension that can be studied separately. That extension is not used to support the quantitative claims in this revision, so the main evidence here concerns the scoring formulation and its end-to-end replay behavior rather than a validated fallback mechanism.

A third implication concerns evaluation methodology. For cache-control methods, it is not enough to assess the quality of an intermediate predictive signal in isolation. What matters is whether the induced eviction decisions improve downstream replay behavior under a controlled simulator. This favors evaluation protocols that report decision-level outcomes, compare against strong practical baselines, and distinguish clearly between offline target quality and online policy quality.

The approach also has broader systems implications. Because the method is based on scoring concrete actions rather than making an all-or-nothing trust decision, it can be paired with heterogeneous predictive context and with the optional guard described above (Section 2.3), subject to the same validation caveat. More generally, the integration of prediction into online systems may depend not only on stronger predictors, but also on learning objectives and control layers that are matched to the structure of the decision being made.

Finally, the framework suggests a broader direction for learning-augmented online decision making. In settings where an algorithm repeatedly chooses among a small set of available actions, candidate-level downstream scoring may provide a more operational route for incorporating predictive information than coarser trust-only formulations. In that sense, the value of the present work may extend beyond paging itself and point toward a broader principle of decision-aligned learning under imperfect predictive information.

4.3. Limitations

The results of this study should be interpreted in light of several limitations. First, the main method is evaluated in the standard unweighted paging setting with fixed cache capacity and unit miss cost. This setting is well suited to isolating the eviction-choice problem and maintaining comparability with established reference policies, but it does not by itself establish effectiveness for weighted paging, variable object sizes, retrieval costs, byte-oriented objectives, or broader general-caching settings.

Second, the proposed supervision target is intentionally local and finite-horizon. The replay-based loss is designed as a tractable approximation to downstream eviction harm rather than as a globally optimal objective. As a result, the learned scorer is trained to predict a local decision proxy rather than an offline-optimal replacement rule. This design is central to the practicality of the method, but it also limits the strength of any claim about long-range optimality.

Third, the end-to-end online replay evaluation in Section 3.4 shows that, at every evaluated capacity (32, 64, and 128), `evict_value_v1` does not improve on LRU, SIEVE, or FIFO-Reinsertion, and that its gap relative to these baselines widens sharply and non-monotonically at capacity 128, in a pattern that is broad-based across four of the six non-degenerate trace families rather than confined to a single outlier. We have not yet isolated the precise causal mechanism behind this capacity-128 finding with a controlled ablation, so the following account should be read as the leading hypothesis rather than a proven explanation. It is consistent with several independent diagnostic checks: the underlying raw outputs are internally consistent and free of duplication or schema drift; the qualitative reversal reproduces deterministically at request counts far smaller than the canonical evaluation, ruling out a large-scale-only artifact; the implementation contains no capacity-specific branching anywhere in the eviction or feature-construction code; and the two most-affected trace families' share of the training data is the smallest among the seven families but is essentially constant across capacities 32, 64, and 128, so a pure training-data-imbalance account would predict a roughly constant relative disadvantage rather than a sudden widening at capacity 128 specifically. The hypothesis we consider most plausible, discussed further in Section 3.6, is a compounding distribution shift between the single-step LRU-continuation counterfactual used to construct the supervision target in Section 2.2 and the policy's actual recursive, non-LRU trajectory once deployed, an effect that plausibly grows with the number of candidates scored per decision and therefore with cache capacity. Confirming this account directly would require either a full-scale targeted replay with per-decision trajectory logging or a small retraining ablation under an alternative continuation rule, both of which we leave to future work. Capacity 256 has not yet been evaluated end-to-end pending further analysis of the capacity-128 finding. Given this evidence, we characterize the present contribution accordingly: `evict_value_v1` is best read as a decision-aligned supervision study that demonstrates a learnable, well-motivated candidate-level target,

not yet as a practically superior online eviction policy, and SIEVE and FIFO-Reinsertion remain strong, low-overhead baselines that the proposed method does not currently outperform end-to-end.

Fourth, the paper is methodological and empirical rather than theorem-centered. We do not provide a new competitive-ratio guarantee for the learned policy, and the fallback mechanism is introduced as a practical control layer rather than as a formally proved safeguard. The contribution should therefore be interpreted as a decision-aligned learning framework for cache replacement, not as a replacement for the classical worst-case guarantees available in online algorithms.

Fifth, empirical conclusions in learning-augmented caching remain sensitive to workload characteristics, cache capacities, and replay protocol choices. Even with heterogeneous trace families and strong baselines, performance can vary across regimes. The current results should therefore be read as evidence that candidate-level finite-horizon scoring is a promising direction under the evaluated settings, rather than as a claim of universal superiority.

Sixth, this revision is the work of a single author, and the validation reported here cannot substitute for independent replication by additional authors or institutions. To mitigate, but not eliminate, this risk, the revision relies on repository-level audit trails, schema and replay-artifact validation checks, and explicit negative-result reporting in place of multi-author cross-checking.

Finally, the fallback variants introduce additional design choices, including the early-return window, trigger threshold, monitoring interval, fallback duration, and fallback policy. These choices are operationally interpretable and easy to implement, but they remain heuristic parameters rather than theoretically derived constants. Their value may therefore depend substantially on workload regime and deployment conditions.

Taken together, these limitations define the intended scope of the paper. The study makes a focused case for candidate-level finite-horizon eviction-value prediction as a practical and decision-aligned direction for learning-augmented caching, while leaving broader cost models, stronger guarantees, and more extensive regime analysis to future work.

4.4. Future Research Directions

Several research directions follow naturally from this study. First, the candidate-centric framework should be extended beyond the unweighted paging setting. While the present paper focuses on unit-cost misses and fixed-

size items in order to isolate the eviction-choice problem, many practical caching systems involve variable object sizes, heterogeneous miss costs, and more general replacement constraints. Extending eviction-value prediction to weighted paging and broader general-caching models would substantially widen the practical scope of the approach.

Second, the learning target itself can be strengthened. The current method uses a finite-horizon replay loss because it provides a tractable and operationally meaningful proxy for downstream eviction quality. Future work could investigate richer targets that better capture longer-range effects, alternative continuation rules, or more structured objectives that reflect broader cache-state interactions. This includes ranking-based or pairwise supervision, provided that such formulations can be shown to improve downstream replay behavior in a stable and reproducible way.

Third, the robustness layer can be made more adaptive. The guarded controller studied here is intentionally lightweight and practical, but it remains heuristic. Future work could develop workload-aware guard policies, disagreement-aware switching criteria, or stronger online detectors for locally unsafe decisions. It would also be useful to study whether fallback logic can be coupled more tightly to uncertainty estimates or diagnostic signals derived from the scorer itself.

Fourth, broader empirical evaluation remains important. Larger benchmark campaigns, wider capacity sweeps, and more detailed slice-level analyses would help clarify where candidate-level eviction scoring is strongest and where robust baselines remain preferable. Continued emphasis on strong baselines, transparent replay protocols, and regime-level analysis would improve both scientific understanding and practical credibility.

Fifth, there is a natural theoretical direction. Although the present paper is not centered on competitive-ratio guarantees, it would be valuable to understand whether restricted forms of candidate scoring or guarded deployment admit meaningful formal guarantees. Even partial theory under simplified assumptions could help bridge the gap between empirically successful learned policies and the robustness standards of online algorithms.

More broadly, the results suggest a general principle for learning-augmented online decision making: when an algorithm repeatedly chooses among a small set of concrete actions, learning the downstream quality of those actions directly may be more effective than relying only on coarse trust decisions about an external hint. Exploring this principle beyond caching may therefore be a fruitful direction for future work.

Acknowledgements

The author thanks Professor Ioannis Koutis for his guidance and support throughout this work. The author also acknowledges the use of the Wulver high-performance computing system at the New Jersey Institute of Technology for part of the experimental evaluation.

Funding

The author received no specific funding for this work.

AI Declaration

During the preparation of this manuscript, the author used ChatGPT for writing assistance and manuscript refinement, and Perplexity AI for literature exploration and background research. During the implementation stage of the project, the author used Cursor, ChatGPT Codex, and GitHub Copilot for coding assistance, and Gemini for limited recommendations, including assistance with auditing repository artifacts and checking internal consistency between the manuscript and the underlying code and results. All AI-assisted text, code, and analysis were independently verified by the author against the repository's artifacts before inclusion, including dedicated schema-validation, replay-artifact, and negative-result audits documented in the repository; AI assistance was not used to introduce any positive result that is not directly supported by these verified artifacts. The author takes full responsibility for the content of the manuscript, the code, and the reported results.

Data Availability

The datasets used in this study are cited in the manuscript and were obtained from their respective public or documented sources. The repository associated with this work contains the code used for data preparation, experiment execution, and analysis: `https://github.com/SoroushVahidi/Augmented-caching`. Access to specific datasets remains subject to the terms, licenses, and availability conditions of the original data providers.

Code Availability

The code used to prepare the data, train the models, run the baseline comparisons, and generate the experimental artifacts is available at: `https://github.com/SoroushVahidi/Augmented-caching`

Declaration of Competing Interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

References

[1] L. A. Belady, A study of replacement algorithms for virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101. doi:10.1147/SJ.52.0078.

[2] A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, N. E. Young, Competitive paging algorithms, Journal of Algorithms 12 (4) (1991) 685–699. doi:10.1016/0196-6774(91)90041-V.

[3] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, Journal of the ACM 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.

[4] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 345–355.

[5] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning (ICML), Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.

[6] A. Wei, Better and simpler learning-augmented online caching, in: Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss

Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.

[7] J. Chłędowski, A. Polak, B. Szabucki, K. Żołna, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.

[8] M. Mitzenmacher, S. Vassilvitskii, Algorithms with predictions, Communications of the ACM 65 (7) (2022) 33–35. doi:10.1145/3528087.

[9] D. Rohatgi, Near-optimal bounds for online caching with machine learned advice, in: Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2020, pp. 1834–1845. doi:10.1137/1.9781611975994.112.

[10] N. Bansal, C. Coester, R. Kumar, M. Purohit, E. Vee, Learning-augmented weighted paging, in: Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2022, pp. 67–89.

[11] A. Blum, C. Burch, On-line learning and the metrical task system problem, Machine Learning 39 (1) (2000) 35–58. doi:10.1023/A:1007621832648.

[12] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL https://proceedings.mlr.press/v119/liu20f.html

[13] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, 2020, pp. 529–544.
URL https://www.usenix.org/conference/nsdi20/presentation/song

[14] X. Hu, E. Ramadan, W. Ye, F. Tian, Z.-L. Zhang, Raven: Belady-guided, predictive (deep) learning for in-memory and content caching,

in: Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '22), ACM, 2022, pp. 72–90. doi:10.1145/3555050.3569134.
URL https://doi.org/10.1145/3555050.3569134

[15] I. Shah, A. Jain, C. Lin, Effective mimicry of belady's MIN policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), IEEE, 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.

[16] Z. Song, K. Chen, N. Sarda, D. Altinbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, Halp: Heuristic aided learned preference eviction policy for youtube content delivery network, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu

[17] A. N. Elmachtoub, P. Grigas, Smart "predict, then optimize", Management Science 68 (1) (2022) 9–26. doi:10.1287/mnsc.2020.3922.

[18] B. Wilder, B. Dilkina, M. Tambe, Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 1658–1665. doi:10.1609/aaai.v33i01.33011658.

[19] G. Tolomei, L. Takanen, F. Pinelli, Mustache: Multi-step-ahead predictions for cache eviction, CoRR abs/2211.02177 (2022).
URL https://arxiv.org/abs/2211.02177

[20] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo

[21] E. Cho, S. A. Myers, J. Leskovec, Friendship and mobility: User movement in location-based social networks, in: Proceedings of the 17th ACM

SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2011, pp. 1082–1090. doi:10.1145/2020408.2020579.

[22] J. L. Henning, Spec cpu2006 benchmark descriptions, ACM SIGARCH Computer Architecture News 34 (4) (2006) 1–17. doi:10.1145/1186736.1186737.

[23] J. Yang, Y. Yue, K. V. Rashmi, A large scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 191–208.
URL https://www.usenix.org/conference/osdi20/presentation/yang

[24] C. A. Waldspurger, N. Park, A. Garthwaite, I. Ahmad, Efficient mrc construction with shards, in: 13th USENIX Conference on File and Storage Technologies (FAST 15), USENIX Association, 2015, pp. 95–110.
URL https://www.usenix.org/conference/fast15/technical-sessions/presentation/waldspurger

[25] Citi Bike, Citi bike system data, accessed: 2026-04-10 (2025).
URL https://citibikenyc.com/system-data

[26] Wikimedia Foundation, Analytics datasets: Pageviews, accessed: 2026-04-10.
URL https://dumps.wikimedia.org/other/pageviews/readme.html

[27] J. Yang, Open-source cache dataset, gitHub repository. Accessed: 2026-04-10 (2024).
URL https://github.com/cacheMon/cache_dataset

[28] CacheLib, Evaluating ssd hardware for facebook workloads, documentation page. Accessed: 2026-04-10.
URL https://cachelib.org/docs/Cache_Library_User_Guides/Cachebench_FB_HW_eval/